ASP.NET Core Out Of Process Hosting

Back to ASP.NET Core Tutorial for Beginners

ASP.NET Core Out of Process Hosting

In this article, I will discuss the ASP.NET Core Out of Process Hosting with Example. Before proceeding, I strongly recommend you read ASP.NET Core In-Process Hosting and Kestrel Web Server in ASP.NET Core Web Application articles.

What is Out of Process Hosting in ASP.NET Core?

In the Out-of-Process Hosting model, the ASP.NET Core application runs in a separate worker process from the web server which proxies the handling the incoming HTTP requests. This is different from the In-Process hosting model, where the application runs within the same worker process as the web server.

With Out-of-Process Hosting model, IIS web server (such as Apache or Nginx server as Linux) stops receiving incoming HTTP requests and then forwarding them to an internal Kestrel. As a result, the ASP.NET Core Middleware pipeline processes the HTTP requests.

- **External Web Server:** The server is commonly IIS (Internet Information Services) on Windows but can also be Apache or Nginx on Linux or macOS. The web server receives HTTP requests from the Internet. The web server then forwards some or all HTTP requests to the hosting web server, keeping other requests, such as static files, and serving them. A benefit to this is that the hosting web server can be configured to forward requests to the Internal Web Server, i.e. Kestrel.
- **Internal Web Server:** The Internal Web Server is Kestrel in the ASP.NET Core application. It runs the ASP.NET Core application, i.e. the Kestrel web server processes the HTTP requests received from the Internet.

This separation allows the internal web server (Kestrel) to be optimized for hosting. ASP.NET Core application code receives only the internal web server traffic and can ignore the external web server's security and hardening features.

How Do We Configure the Out-Of-Process Hosting in ASP.NET Core Application?

We can configure the out-of-process hosting in two ways in ASP.NET Core.

Method 1: Using Project Properties File

In this case, we need to set the **<AspNetCoreHostingModel>** declared in the application project file with a value **OutOfProcess**. So, open the Project Properties file and add the following code.

```
<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>
```

Once we click on the Save button, Visual Studio will automatically update the project file and open. From the window, select the Debug tab, and then click on the Open debug launch profiles UI link. Now, we can see the Kestrel and IIS Express profiles. By default, we select the Kestrel profile as the hosting model. In the Project Properties window, the Out of Process Hosting Model is selected as the default hosting model.

Method 2: Specify the Hosting Model on OutOfProcess using CLI

To do so, right-click on your project in the Solution Explorer and click on the Properties option from the context menu, which will open the Project Properties window. Now, select the Debug tab from the window and click on the Open debug launch profiles UI link, as shown in the image below.

How Do We Use Out-Of-Process Hosting in ASP.NET Core?

Kestrel can run as the ASP.NET CORE publish.exe in out-of-process, neither the IIS/Kestrel nor the OutOfProcess hosting model will be installed. In that case, Kestrel is the only web server that will listen and run the application. The IIS/Kestrel web server will not listen to the incoming HTTP requests, which will be sent to Kestrel by the OutOfProcess hosting model.

Kestrel can run as the ASP.NET CORE publish.exe in out-of-process, neither the IIS/Kestrel nor the OutOfProcess hosting model will be installed. In that case, Kestrel is the only web server that will listen and run the application. The IIS/Kestrel web server will not listen to the incoming HTTP requests, which will be sent to Kestrel by the OutOfProcess hosting model.

In this scenario:

- Kestrel Web Server will handle both incoming HTTP requests and process the application code.
- Kestrel will act as the Internet-facing web server, receiving requests directly from the Internet, processing them by executing the application code, and returning the response to the requesting client.
- Kestrel will run as a standalone HTTP server, listening on localhost, or Nginx or Apache. Internet-facing Kestrel serves both as the application host and the HTTP server.

Kestrel as Internal Proxy Web Server (External Web Server)

Suppose the application is not directly using HTTP or HTTPS protocols, then Kestrel Web Server will act as an internal web server. In this case, Kestrel is not the web server that will listen and run the application. The IIS/Kestrel web server will listen to the incoming HTTP requests, which will be sent to Kestrel by the OutOfProcess hosting model.

In this scenario:

- Kestrel Web Server will handle both incoming HTTP requests and process the application code.
- Kestrel will act as the Internal-facing web server, receiving requests directly from the Internet, processing them by executing the application code, and returning the response to the requesting client.
- Kestrel will run as a standalone HTTP server, listening on localhost, or Nginx or Apache. Internal-facing Kestrel serves both as the application host and the HTTP server.

Windows Process Hosting in Kestrel

The worker process that runs Kestrel depends on the deployment mode:

- If the application is deployed as Framework-Dependent, the worker process runtime will be dotnet.exe. This means the application runs on the .NET Core runtime installed and shared on the system.
- If the application is deployed as Self-Contained, the worker process runtime will be the application's exe. For example, if the project name is FirstCoreWebApplication, the worker process name will be FirstCoreWebApplication.exe (Windows).

Run the application using HTTP/HTTPS Launch Profile

Let's provide the Kestrel Hosting Model in the Project.

```
{
  "profiles": {
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "hotReloadProfile": "aspnetcore"
    },
    "FirstCoreWebApplication": {
      "commandName": "Project",
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7194;http://localhost:5194",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "hotReloadProfile": "aspnetcore"
    }
  }
}
```

Now, run the application using either the http or https profile and select the worker process name as shown in the image below. Here, it shows FirstCoreWebApplication, meaning Kestrel Server executes our application code.

The process that when we run the HTTP or HTTPS profile to run our application in Visual Studio, Kestrel is the only Web Server that hosts our application, handles the incoming HTTP Requests, and executes the application code. In this case, Kestrel is not acting as a proxy server.

Kestrel as Internal Web Server

Kestrel can also act as the Internal Web Server. It can be used with a Reverse Proxy (Internal Facing Server) such as IIS, Apache, or Nginx. Now the question that should come to your mind is, if Kestrel can be used as a Web Server that can handle incoming HTTP requests, then why do we need a Reverse Proxy Server?

Kestrel is a very fast and secure web server, but it is not a full-featured web server. The Reverse Proxy Server can offer many benefits to the application that Kestrel does not provide. The Kestrel Internal Web Server can be used as the Internal Web Server, receiving requests from the Reverse Proxy Server, and then sends the response back to the requesting client via the Reverse Proxy Server.

How Does the Out-Of-Process Hosting Model Work?

The Out-of-Process Hosting Model comes into play when an ASP.NET Core application is hosted using IIS or other web servers such as Apache or Nginx. In this case, the application runs in a separate process. The external web server acts as a reverse proxy, forwarding requests to the internal Kestrel server.

Let's understand how the Out-of-Process Hosting Model works.

Step 1: Internal User's Request:

This is where the user interacts with the application. The process begins with the client making an HTTP request to the server. The request is initiated by the user's browser or any HTTP client and is directed to the server hosting the application. The request travels through the Internet or other network infrastructure and is received by the server on the machine it is hosted on.

Step 2: IIS Internal Integration Services:

IIS is the external web server that receives the incoming requests. IIS acts as a reverse proxy server and uses the IIS Internal Integration Services to forward the requests to the internal Kestrel server.

Step 3: ASP.NET Core Middleware:

The ASP.NET Core Middleware (ANCM) is a component within IIS that manages interaction between IIS and the Kestrel server. It ensures that requests are properly forwarded from IIS to the ASP.NET Core application and Kestrel processing requests is Kestrel's responsibility.

- ANCM checks if the Kestrel server is up and running.
- If the Kestrel server is not running, ANCM detects it.
- Once Kestrel starts up, ANCM forwards the HTTP request to Kestrel for further processing.

Step 4: Kestrel Web Server:

Kestrel is a lightweight, high-performance web server. It handles the HTTP requests forwarded by ANCM. Here is what happens:

- Kestrel receives the HTTP request from ANCM.
- Kestrel processes the HTTP request and forwards it to the ASP.NET Core application, which handles the business logic and generates the response.

Step 5: Application Code (ASP.NET Core Application):

This is where the ASP.NET Core application logic is executed. The application code includes controllers, services, and middleware components to handle the request. The ASP.NET Core Middleware Pipeline processes the request. Some tasks handled at this stage are:

- **Authentication/Authorization:** Verifying whether the user has access to the requested resource.
- **Data Access:** Interacting with databases or other data sources to retrieve data.
- **Business Logic:** Executing any necessary business logic, interacting with services or databases.

Once the processing is complete, the application generates a response (such as an HTML page, JSON data, or other content). Once the response is ready, the Application Code returns to the Kestrel server.

Step 6: Kestrel Web Server:

Once Kestrel receives the response, it forwards the response to the ASP.NET Core Middleware (ANCM).

Step 7: ASP.NET Core Middleware (ANCM):

Once the ASP.NET Core Middleware receives the response, it does not modify it. Instead, it simply forwards it to IIS.

Step 8: IIS:

IIS receives the response from the ASP.NET Core Middleware (ANCM) and prepares it to be sent back to the client who initiated the request.

Step 9: Internet (Client's Browser):

The response travels through the Internet or other network infrastructure to the client's browser or client application, where it can be processed and displayed to the user.

Kestrel as an Internal Web Server:

Let's see how to configure the Kestrel Web Server to process all the requests on the application using the IIS Express profile. As shown in the image below, you will see the worker process as your project name. You can also verify in the Kestrel that the Kestrel is running.

With the Out-of-Process Hosting Model, when we use IIS Express as the Kestrel profile to run the application, IIS Express acts as the Reverse Proxy Server and Kestrel Web Server runs and listens on the localhost.

Now, IIS Express receives the incoming HTTP request and then forwards it to the Kestrel Web Server for processing. The Kestrel Web Server processes the request, generates the response, and sends it back to IIS Express, which in turn sends the response back to the client, i.e. to the browser.

What is the Role of IIS Express Web Server?

The IIS Express Proxy Server, implemented by iisexpress.exe, acts as a reverse proxy for ASP.NET Core applications. Its primary functions include:

- **Handling Incoming Requests:** Receives HTTP/HTTPS requests from clients.
- **Security and Authentication:** Manages initial security tasks, such as handling SSL/TLS encryption and authentication (e.g., Windows Authentication).
- **Load Balancing:** Distributes incoming requests among multiple worker processes (like IIS), ensuring efficient resource utilization.
- **URL Rewriting and Routing:** It can perform URL rewriting or routing based on predefined rules to the correct application instance, facilitating multi-tenancy or microservices architecture within a single server.
- **Load Balancing:** It can distribute requests among multiple worker processes (like IIS) to ensure efficient resource utilization.
- **Logging and Monitoring:** Provides logs and monitoring data for troubleshooting and performance analysis.

(IIS) performing these tasks ensures that the Kestrel Web Server focuses on processing the application code. Separating the ASP.NET Core Middleware (ANCM) for application-level processing.

What is the Role of Kestrel Web Server Process?

The Kestrel Web Server is a cross-platform, high-performance web server for processing requests in an ASP.NET Core application. Its primary functions include:

- **Application Logic Execution:** Once Kestrel receives the request from IIS or the ASP.NET Core Middleware, it passes it to the appropriate middleware components in the ASP.NET Core application. This includes routing, authentication, logging, implementation, routing, and response handling.
- **Response Generation:** The application logic processes the request and generates a response, including HTML, JSON, or other data. Kestrel ensures that the response is formatted correctly and sent back to the middleware for further processing.
- **Communication with IIS:** Kestrel communicates with IIS to receive requests and send responses. Kestrel sends the response back through the ASP.NET Core Middleware to IIS, which then transmits it back to the client.

In the next article, I will discuss the ASP.NET Core Load Balancing Options with Examples. In this article, I explain the ASP.NET Core Out of Process Hosting Model with examples. I hope this article will help you to understand the ASP.NET Core Out of Process Hosting Model in ASP.NET Core Web Application.

Join Net Tutorials
Share the Author's Profile Page
Please Pick the preferred date-time CDMY dmmm hh:mm:ss yyyy zzzzz Pacific/Kiritimati Pacific/Pago_Pago Pacific/Midway Pacific/Niue Pacific/Honolulu Pacific/Rarotonga Pacific/Tahiti Pacific/Marquesas America/Adak Pacific/Gambier Pacific/Pitcairn America/Anchorage America/Juneau America/Nome America/Sitka America/Yakutat America/Metlakatla America/Los_Angeles America/Tijuana America/Vancouver America/Whitehorse America/Dawson America/Dawson_Creek America/Fort_Nelson America/Boise America/Cambridge_Bay America/Chihuahua America/Ciudad_Juarez America/Creston America/Dawson America/Denver America/Edmonton America/Eirunepe America/Fort_Nelson America/Glace_Bay America/Goose_Bay America/Halifax America/Inuvik America/Iqaluit America/Kralendijk America/Lower_Princes America/Maceio America/Manaus America/Marigot America/Martinique America/Moncton America/Menominee America/Metlakatla America/Nipigon America/Pangnirtung America/Rainy_River America/Resolute America/Santarem America/Thule America/Yellowknife America/Yellowknife America/Winnipeg America/Ciudad_Juarez America/Cuiaba America/Danmarkshavn America/Edmonton America/Eirunepe America/Fortaleza America/Glace_Bay America/Godthab America/Goose_Bay America/Grand_Turk America/Guayaquil America/Guyana America/Halifax America/Havana America/Hermosillo America/Indianapolis America/Inuvik America/Iqaluit America/Jamaica America/Kralendijk America/La_Paz America/Lima America/Los_Angeles America/Maceio America/Manaus America/Marigot America/Martinique America/Matamoros America/Mazatlan America/Menominee America/Metlakatla America/Mexico_City America/Miquelon America/Moncton America/Monterrey America/Montevideo America/Nipigon America/Nome America/Noronha America/Nuuk America/Ojinaga America/Pangnirtung America/Paramaribo America/Phoenix America/Port-au-Prince America/Porto_Velho America/Puerto_Rico America/Rainy_River America/Rankin_Inlet America/Resolute America/Santarem America/Santiago America/Santo_Domingo America/Sao_Paulo America/Scoresbysund America/Sitka America/Thule America/Tijuana America/Toronto America/Tortola America/Vancouver America/Whitehorse America/Winnipeg America/Yakutat America/Yellowknife Antarctica/Casey Antarctica/Davis Antarctica/DumontDUrville Antarctica/Macquarie Antarctica/Mawson Antarctica/McMurdo Antarctica/Palmer Antarctica/Rothera Antarctica/Syowa Antarctica/Troll Antarctica/Vostok Asia/Aden Asia/Almaty Asia/Amman Asia/Anadyr Asia/Aqtau Asia/Aqtobe Asia/Ashgabat Asia/Atyrau Asia/Baku Asia/Bangkok Asia/Barnaul Asia/Beirut Asia/Bishkek Asia/Brunei Asia/Chita Asia/Choibalsan Asia/Colombo Asia/Damascus Asia/Dhaka Asia/Dili Asia/Dubai Asia/Dushanbe Asia/Famagusta Asia/Gaza Asia/Hebron Asia/Ho_Chi_Minh Asia/Hovd Asia/Irkutsk Asia/Jakarta Asia/Jayapura Asia/Jerusalem Asia/Kabul Asia/Kamchatka Asia/Karachi Asia/Kathmandu Asia/Khandyga Asia/Kolkata Asia/Krasnoyarsk Asia/Kuala_Lumpur Asia/Kuching Asia/Macau Asia/Magadan Asia/Makassar Asia/Manila Asia/Nicosia Asia/Novokuznetsk Asia/Novosibirsk Asia/Omsk Asia/Oral Asia/Pontianak Asia/Pyongyang Asia/Qatar Asia/Qyzylorda Asia/Riyadh Asia/Sakhalin Asia/Samarkand Asia/Seoul Asia/Shanghai Asia/Singapore Asia/Srednekolymsk Asia/Taipei Asia/Tashkent Asia/Tbilisi Asia/Tehran Asia/Thimphu Asia/Tokyo Asia/Tomsk Asia/Ulaanbaatar Asia/Urumqi Asia/Ust-Nera Asia/Vientiane Asia/Vladivostok Asia/Yakutsk Asia/Yangon Asia/Yekaterinburg Asia/Yerevan Atlantic/Azores Atlantic/Bermuda Atlantic/Canary Atlantic/Faroe Atlantic/Madeira Atlantic/Reykjavik Atlantic/South_Georgia Atlantic/St_Helena Atlantic/Stanley Australia/Adelaide Australia/Brisbane Australia/Broken_Hill Australia/Darwin Australia/Eucla Australia/Hobart Australia/Lindeman Australia/Lord_Howe Australia/Melbourne Australia/Perth Australia/Sydney Europe/Amsterdam Europe/Andorra Europe/Astrakhan Europe/Athens Europe/Belgrade Europe/Berlin Europe/Bratislava Europe/Brussels Europe/Bucharest Europe/Budapest Europe/Busingen Europe/Chisinau Europe/Copenhagen Europe/Dublin Europe/Gibraltar Europe/Guernsey Europe/Helsinki Europe/Isle_of_Man Europe/Istanbul Europe/Jersey Europe/Kaliningrad Europe/Kiev Europe/Kirov Europe/Lisbon Europe/Ljubljana Europe/London Europe/Luxembourg Europe/Madrid Europe/Malta Europe/Mariehamn Europe/Minsk Europe/Monaco Europe/Moscow Europe/Oslo Europe/Paris Europe/Podgorica Europe/Prague Europe/Riga Europe/Rome Europe/Samara Europe/San_Marino Europe/Sarajevo Europe/Saratov Europe/Simferopol Europe/Skopje Europe/Sofia Europe/Stockholm Europe/Tallinn Europe/Tirane Europe/Tiraspol Europe/Ulyanovsk Europe/Uzhgorod Europe/Vaduz Europe/Vatican Europe/Vienna Europe/Vilnius Europe/Volgograd Europe/Warsaw Europe/Zagreb Europe/Zaporozhye Europe/Zurich Indian/Antananarivo Indian/Chagos Indian/Christmas Indian/Cocos Indian/Comoro Indian/Kerguelen Indian/Mahe Indian/Maldives Indian/Mauritius Indian/Mayotte Indian/Reunion Pacific/Apia Pacific/Auckland Pacific/Bougainville Pacific/Chatham Pacific/Chuuk Pacific/Easter Pacific/Efate Pacific/Fakaofo Pacific/Fiji Pacific/Funafuti Pacific/Galapagos Pacific/Gambier Pacific/Guadalcanal Pacific/Guam Pacific/Kanton Pacific/Kiritimati Pacific/Kosrae Pacific/Kwajalein Pacific/Majuro Pacific/Marquesas Pacific/Nauru Pacific/Niue Pacific/Norfolk Pacific/Noumea Pacific/Pago_Pago Pacific/Palau Pacific/Pitcairn Pacific/Pohnpei Pacific/Port_Moresby Pacific/Rarotonga Pacific/Saipan Pacific/Tahiti Pacific/Tarawa Pacific/Tongatapu Pacific/Wake Pacific/Wallis Pacific/Yap Pacific/Chuuk Pacific/Pohnpei Pacific/Kosrae